Familiar calls.

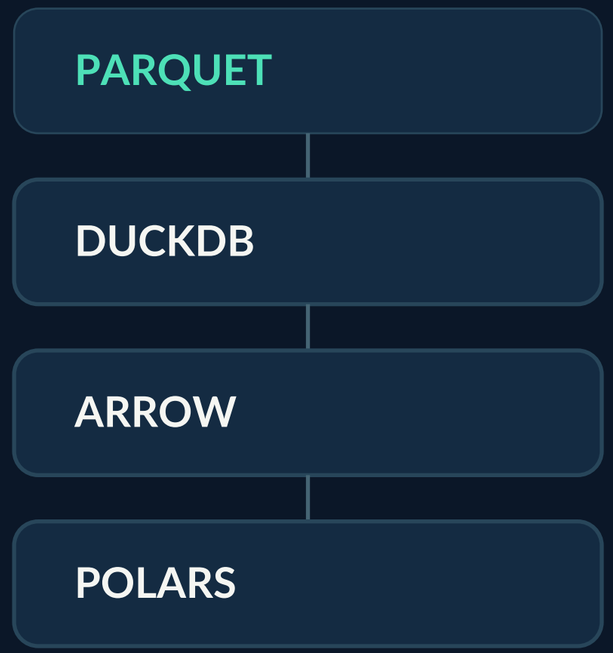
Modern analytics.

intermine314

0.2.0

Measured against original InterMine 1.13.0: speed, memory, storage and the cost of migration.

CURRENT SOURCE PREVIEW — NOT A NEW PYPI RELEASE



The largest gains arrive when data is reused.

One million annotation rows, eight columns. Five fresh-process repetitions per workflow.

32.2×

Faster repeated aggregation

CSV + pandas versus Parquet + DuckDB. Warm OS cache; same grouped result.

97%

Less disk than plain CSV

Zstandard Parquet versus streamed CSV. Data entropy and compression change the saving.

38%

Lower export peak RSS

Four-worker Parquet versus original dataframe → CSV. Streaming CSV has a lower memory floor.

These are measured workflow comparisons, not a universal package speedup. Small jobs and direct CSV export can favor the original client.

Keep the familiar entry point.

The historical facade restores legacy behavior around a shared modern runtime.

```
from intermine314.webservice import Service
service = Service(mine_url)
query = service.new_query("Gene")
query.add_view("Gene.primaryIdentifier")
for row in query.rows(): ...
```

338 passing scopes

122 tested departures

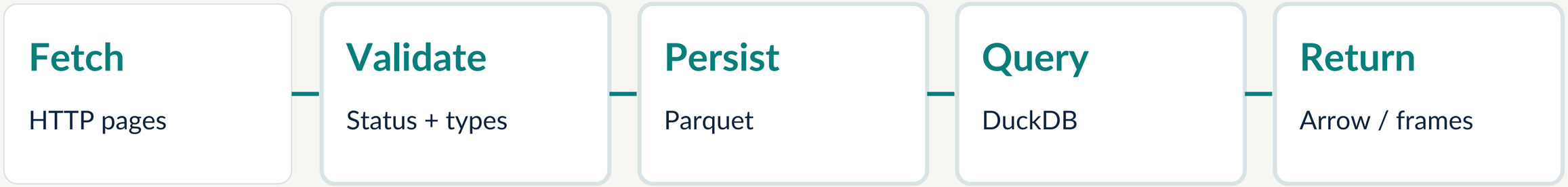
460 inventoried symbols

Queries, constraints, templates, rows, lists and metadata retain familiar entry points. Legacy len(), truth checks and mutable row metadata retain their expected behavior.

For query.dataframe(), install the pandas extra. Output uses Arrow dtypes. The audit counts are scoped assertions, not a promise of complete API equivalence.

One data pipeline behind both interfaces.

A modern storage workflow is explicit; import replacement alone does not activate every optimization.



Bounded pages and typed Arrow conversion avoid retaining every result as Python dictionaries. Parquet becomes the reusable data source; SQL pushes filtering and aggregation into DuckDB.

Polars frames by default in native mode. Optional pandas at the legacy boundary. Explicit ownership closes streams, readers and connections.

Compatibility and native modes have different costs.

Warm local fixtures, same Python 3.14.5. No HTTP, storage or native-buffer RSS in these timings.

Operation	Original	Legacy facade	Native	Import-level gain
200 XML serializations	5.14	2.11	2.21	2.4×
10k streamed dictionaries	16.67	13.75	7.57	1.2×
10k dictionaries → list	33.00	27.50	7.81	1.2×
10k retained row objects	14.03	12.51	5.56	1.1×

Legacy list() can issue two requests, matching original behavior. Native iterators are unsized and consume one request; native gains require choosing that interface.

CSV is cheap to write. Parquet pays for future reads.

HTTP response → output file, including decoding and conversion. No artificial network latency.

Workflow	100k rows	1M rows	1M requests
Original → streaming CSV	0.248	2.441	1
Original → pandas → CSV	0.713	7.328	1
intermine314 → Parquet / 1 worker	0.405	2.756	100
intermine314 → Parquet / 4 workers	0.418	2.864	100

Original CSV paths use one streaming response. Parquet exports use bounded 10k-row pages and atomic publication. This compares complete workflows, not identical storage formats.

Memory savings depend on data shape.

Peak process RSS in MiB includes Python objects, imported engines and native buffers.

Export workflow	100k annot.	1M annot.	1M diverse
Original → streaming CSV	79.2	79.1	79.2
Original → pandas → CSV	158.6	852.7	727.3
intermine314 → Parquet / 1 worker	288.7	528.6	784.6
intermine314 → Parquet / 4 workers	298.5	532.0	778.2

For diverse 1M rows, four-worker Parquet used 7% more RSS than original pandas. Streaming CSV stayed smallest. Queue byte estimates bound admitted work, not total process RSS.

Compressible annotations benefit most.

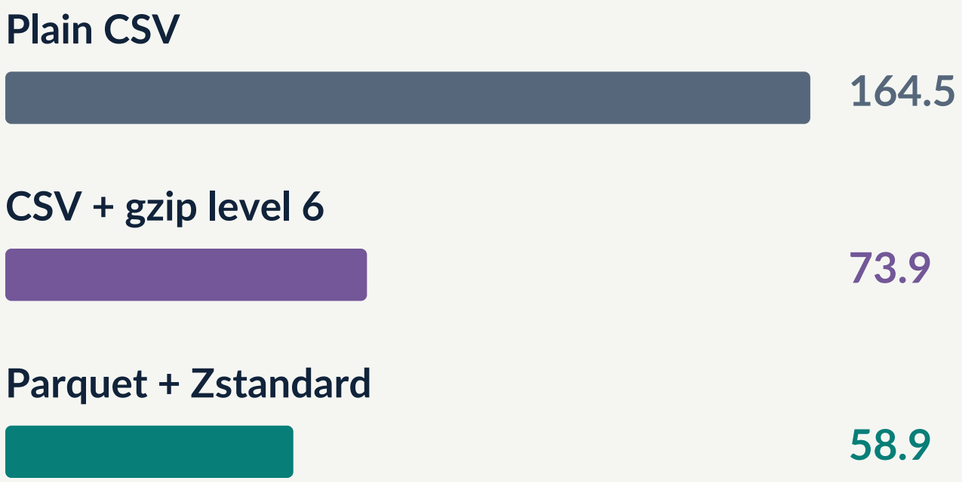
The same one million typed records per format. Every persisted value was checked after decoding.

Repeated annotations



96.7% smaller than plain CSV

Diverse identifiers + sequences



64.2% smaller than plain CSV

gzip is a meaningful control. Parquet also retains schema and supports column selection; compression alone does not explain its value.

Read only the columns needed for the question.

Group by chromosome; count records and sum lengths. Same result, warm filesystem cache.

Read + aggregate	Annotations	Diverse data	Annotations RSS
CSV + Python streaming	0.968	1.043	25.8 MiB
CSV + pandas / two columns	0.635	0.333	115.9 MiB
CSV + DuckDB / control	0.114	0.126	294.9 MiB
Parquet + package DuckDB	0.020	0.019	158.4 MiB

The DuckDB-over-CSV control separates engine improvements from file-format improvements. A new connection is included per operation; imports and interpreter startup are excluded.

Repeated analysis can repay the conversion cost.

Original pandas/CSV vs Parquet/DuckDB: export + N analyses. Modeled totals, not a live-mine forecast.

Dataset	1 analysis	10 analyses	100 analyses
Annotations / 100k	+0.35 s	+0.84 s	+5.74 s
Annotations / 1M	+5.08 s	+10.62 s	+66.01 s
Diverse / 100k	+0.22 s	+0.43 s	+2.55 s
Diverse / 1M	+4.12 s	+6.94 s	+35.17 s

Positive values are time saved; negative values mean the modern workflow took longer. Network conditions, cold storage, query shape and concurrent load can change these estimates.

A 16-worker budget is shared across stages.

Profiles cpu16, cpu8, cpu6, cpu4 and cpu2 adapt to available CPUs and memory.

Stage	Starting allocation	Design constraint
Public HTTP / metadata	4 / 4	Per-host permits and bounded pages
Arrow CPU / I/O	4 / 2	Convert bounded batches
Ordered / unordered COPY	4 / 8	One copy; order only when required
DuckDB SQL / Polars	16 / 12	One main plan; shared admission

Observed 1 → 4 HTTP workers: 0.96× on the 1M annotation fixture. More threads are not automatically faster. Host: Ultra 7 265K, WSL2 exposes 18 CPUs, not all 20 cores.

Performance improvements retain validation.

The export is accepted only after its pages and final schema are valid.

Validate every page

Read terminal server status before accepting rows. Failures leave existing output intact.

Account for waiting results

Completed pages waiting for earlier pages stay charged to outstanding-page and byte estimates.

Decode compressed metadata

Sized and whole-response reads decode supported compression consistently.

Preserve typed values

Arrow and Parquet preserve explicit types; unsafe precision conversions fail instead of silently rounding.

Evidence supports a scoped replacement.

The core pipeline and historical facade share tests; the compatibility boundary remains explicit.

2,805

Source tests passed

Combined extras on Python 3.14.5. Linux CI; no claim of exhaustive platform coverage.

1,003

Installed regressions

Executed outside the checkout against the wheel. Seven independent install modes verified.

2 mines

Live read-only smoke

LegumeMine and MaizeMine: serial/parallel values, exports and template parsing. Small samples only.

Not certified here: free-threaded Python, subinterpreters, Windows/macOS, authenticated writes or all legacy argument combinations. Pandas uses Arrow dtypes; native iterators are unsized.

Start with the facade. Persist data when it will be reused.

Keep existing query-building code; adopt columnar storage where its measured benefits apply.

```
from intermine314.webservice import Service
from intermine314.export import query_parquet
query = Service(mine_url).new_query("Gene")
query.add_view("Gene.primaryIdentifier")
query.to_parquet("genes.parquet", single_file=True)
frame = query_parquet("genes.parquet", "SELECT count(*) FROM results")
```

Use the pandas extra for legacy dataframe() calls. Review documented departures and resource settings before moving large workflows.

Publication is deferred. Existing PyPI 0.2.0 is an earlier artifact; the benchmarked code is a prepared source build.

Measured, reproducible and explicit about limits.

No comparison in this deck uses intermine314 0.1.7 as its performance baseline.

[B1] Complete workflow trials, storage and method

[B2] Original / legacy / native API trials

[A1] Compatibility coverage and departures

[A2] Runtime resource policy

Both clients use CPython 3.14.5. Original InterMine 1.13.0 needs a benchmark-only MutableMapping alias to import Service. No installed package source was patched. Candidate includes its optional speed extra.

Synthetic 100k / 1M rows; two entropy profiles; five isolated runs; randomized stage order. Full-value hashes checked. Warm OS cache, no fsync, no WAN. RSS includes native memory. Python-version gains are not isolated.